

Laboratorio

Cliente TCP con Raw Sockets

Modificación, análisis y visualización de una sesión TCP

Objetivo

A partir del cliente TCP proporcionado, que construye manualmente los headers IP y TCP mediante raw sockets, el estudiante deberá realizar modificaciones sobre el programa para permitir una interacción más completa con el usuario y mejorar la información mostrada durante la ejecución.

El laboratorio base ya implementa correctamente:

- Creación del raw socket.
- Construcción manual de headers IPv4 y TCP.
- Three-way handshake.
- Envío de datos mediante TCP.
- Recepción de la respuesta del servidor.
- Recepción y procesamiento del FIN enviado por el servidor.
- Cierre de la conexión iniciado posteriormente por el cliente.
- Cálculo de checksums IP y TCP.

La asignación consiste en comprender la implementación existente y modificarla sin alterar el funcionamiento fundamental del protocolo TCP.

Código base

Archivo	Responsabilidad
tcp_client.c	Interfaz con el usuario, argumentos de línea de comandos y control de la sesión
tcp_session.c	Implementación de la sesión TCP, handshake, envío de datos y cierre
tcp_session.h	Definiciones relacionadas con la sesión TCP
tcp_header.c/h	Construcción del header TCP
ip_header.c/h	Construcción del header IPv4
checksum.c/h	Cálculo de checksums
raw_socket.c/h	Creación, envío y recepción mediante raw sockets
ServerTCP.java	Servidor utilizado para realizar las pruebas

El estudiante debe trabajar principalmente sobre: tcp_client.c y tcp_session.c

No se requiere modificar la implementación del servidor para cumplir con los requisitos principales de esta tarea.

Funcionamiento actual

La implementación proporcionada establece una sesión TCP siguiendo una secuencia controlada manualmente.

===== 3 WAY HANDSHAKE =====

1. Cliente → Servidor : SYN
2. Servidor → Cliente : SYN + ACK
3. Cliente → Servidor : ACK

===== DATA =====

4. Cliente → Servidor : PSH + ACK + DATA
5. Servidor → Cliente : ACK
6. Servidor → Cliente : PSH + ACK + DATA
7. Cliente → Servidor : ACK

===== CIERRE DE CONEXIÓN =====

=== Opción 1 ===

8. Servidor → Cliente : FIN + ACK
9. Cliente → Servidor : ACK
10. Cliente → Servidor : FIN + ACK
11. Servidor → Cliente : ACK

=== Opción 2 ===

8. Cliente → Servidor : FIN + ACK
9. Servidor → Cliente : ACK
10. Servidor → Cliente : FIN + ACK
11. Cliente → Servidor : ACK

Importante

- El orden del cierre no debe modificarse.
- Validar que opción de cierre de conexión está realizando el código proporcionado.
- La secuencia debe poder observarse en el Log del laboratorio al igual que en Wireshark.

Requerimiento 1: Permitir mensajes personalizados

Actualmente el cliente utiliza un mensaje por defecto:

EXIT OFF

El estudiante deberá modificar el programa para que el usuario pueda interactuar con el cliente y enviar mensajes personalizados.

Por ejemplo:

- Ingrese un mensaje: Hello World
- El cliente deberá enviar: "Hello World" al servidor.
- Posteriormente deberá poder realizarse otra ejecución utilizando:
- Ingrese un mensaje: "Hola desde Guatemala" o cualquier otro mensaje.
Conservar si se envía el mensaje "EXIT OFF" que termine la ejecución.

Restricciones

- El programa debe continuar realizando correctamente la secuencia en Wireshark.
- El cambio solicitado no debe romper la implementación del protocolo TCP existente.
- La solución deberá permitir que el usuario proporcione el mensaje sin necesidad de modificar el código fuente cada vez.
- Se tomará como mínimo válido si puede enviar 2 mensajes sin opción a más, pero la implementación completa es que pueda enviar N cantidad de mensajes.
- La decisión sobre cómo implementar esta interacción queda a criterio del estudiante, siempre que cumpla el objetivo.

Requerimiento 2: Mejorar el output de consola

El programa actualmente muestra información básica sobre las operaciones realizadas. El estudiante deberá modificar el programa para mostrar en consola los datos más relevantes de cada segmento TCP que participa en la sesión.

La información debe permitir identificar claramente:

- Quién envía el segmento.
- Quién recibe el segmento.
- Flags TCP.
- Número de secuencia (SEQ).
- Número de reconocimiento (ACK).
- Longitud de los datos (LEN).
- Datos enviados o recibidos cuando corresponda.

Formato esperado

El formato exacto queda a criterio del estudiante, pero la salida debe ser clara y permitir seguir la sesión TCP.

Como referencia, una ejecución podría verse de la siguiente manera:

```
===== TCP SESSION =====
Client → Server [SYN]      SEQ=1000
Server → Client [SYN, ACK] SEQ=5000 ACK=1001
Client → Server [ACK]      SEQ=1001 ACK=5001
===== DATA =====
Client → Server [PSH, ACK] SEQ=1001 ACK=5001 LEN=9 DATA="EXIT OFF\n"
Server → Client [ACK]      SEQ=5001 ACK=1010
Server → Client [PSH, ACK] SEQ=5001 ACK=1010 LEN=9 DATA="EXIT OFF\n"
Client → Server [ACK]      SEQ=1010 ACK=5010
===== CLOSE =====
Server → Client [FIN, ACK] SEQ=5010 ACK=1010
Client → Server [ACK]      SEQ=1010 ACK=5011
Client → Server [FIN, ACK] SEQ=1010 ACK=5011
Server → Client [ACK]      SEQ=5011 ACK=1011
===== CONNECTION CLOSED =====
```

El objetivo es que el output permita reconstruir la sesión TCP sin necesidad de revisar inmediatamente Wireshark.

Valide que su secuencia sea parecida a la que se muestra en Wireshark.

NOTA

La inteligencia artificial puede utilizarse como apoyo durante el desarrollo de la tarea. Sin embargo, su objetivo debe ser ayudar al estudiante a comprender el código, no sustituir el proceso de aprendizaje.

Por ejemplo, es válido utilizar un asistente de inteligencia artificial para preguntar:

¿Qué hace esta función?

o:

¿Por qué aquí se incrementa SEQ?

o:

Explícame qué ocurre cuando recibimos un FIN.

También puede utilizarse para analizar un error, interpretar una captura de Wireshark o comprender una función específica.

No se recomienda utilizar la inteligencia artificial simplemente para solicitar: Dame el código completo de la tarea y copiar el resultado sin comprenderlo.

Entregable

Código fuente

Los archivos modificados principalmente:

- tcp_client.c
- tcp_session.c

y cualquier otro archivo que haya sido necesario modificar.

2. README_Lab.md

Debe explicar:

- Qué modificaciones realizó.
- Cómo funciona la interacción con el usuario.
- Cómo se genera el output.
- Qué información TCP muestra el programa.
- Cómo verificó el funcionamiento mediante Wireshark.

3. Captura de Wireshark

- Debe incluir una captura donde pueda observarse la sesión completa:

Todo esto en un ZIP por el GES, debe de grabar un video mostrando y explicando lo anterior.

- La calificación del laboratorio es presencial, si no es posible el video servirá como apoyo en la calificación.